RESEARCH PROJECT OVERVIEW

Adaptive Multi-Agent & Explainable AI

System for Real-Time API Threat Detection

A Research Proposal for Team Review

1. The Problem We Are Solving

Modern applications expose APIs — the communication channels between services, apps, and users. These APIs are increasingly being attacked. But existing security tools are failing in three key ways:

❌ Too Many False Alarms	❓ No Explanation Given	🐢 Too Slow to be Useful
Single-model systems flag innocent traffic, blocking real users and hurting business.	Security teams cannot understand why a request was blocked — making investigation impossible.	Deep analysis of every request adds unacceptable delays in real-time systems.

What Is Missing in Current Research?

Existing ensemble (multi-model) methods combine predictions using simple majority voting — every model gets an equal say, regardless of how confident it is. No existing system combines all of the following:

What Exists Today	What Our System Adds
Simple majority voting (all models equal)	✅ **Confidence-aware weighted voting**
Single-purpose detection models	✅ **3 specialized agents working in parallel**
Analyze every request equally	✅ **Smart filtering — only analyze suspicious traffic**
Black-box decisions	✅ **Plain-English explanations for every decision**

2. Our Proposed Solution

Core Idea

Instead of one AI model making a decision alone, we deploy three specialized AI agents — each expert in a different type of threat — and combine their findings using a smart voting system that weighs each agent by how confident it is. Suspicious requests are automatically flagged with a plain-English explanation of why they were blocked.

Research Objectives

Objective	Description
Objective 1	Design a multi-agent AI system where 3 specialized models detect API attacks in parallel
Objective 2	Implement confidence-aware weighted voting to improve accuracy over simple majority voting
Objective 3	Achieve sub-200ms response time using a smart fast-filter that bypasses AI for normal traffic
Objective 4	Generate human-readable explanations for every security decision (Explainable AI)
Objective 5	Detect both known attacks AND novel (zero-day) attack variations

3. How the System Works

Every API request passes through the following pipeline. The system is designed so that only suspicious traffic reaches the AI agents — keeping the system fast for normal users.

1	**Request Arrives at API Gateway** An external client (app, service, or user) sends an HTTP request. The gateway extracts key features: headers, payload content, IP address, timestamps, and request frequency.
▼	
2	**Fast Filter (Rule-Based Check)** A lightweight rule engine checks the request in under 5ms. Roughly 80% of normal traffic passes through instantly — only suspicious or unknown requests continue to AI analysis.
▼	
3	**Parallel AI Agent Analysis** Three specialized AI agents analyze the suspicious request simultaneously — each expert in a different type of threat. This is the core of the system (detailed in the next section).
▼	
4	**Decision Engine — Confidence-Weighted Voting** A decision engine collects scores from all three agents. Each agent's vote is weighted by its confidence level. A final threat score is calculated and the request is classified as: ALLOW, FLAG, or BLOCK.
▼	
5	**Explanation Generated** The Explainable AI layer produces a human-readable explanation — which agent raised the alarm, what suspicious pattern was found, and what the recommended action is.
▼	
6	**Response Sent + Dashboard Updated** The security decision is sent back to the client. The result is logged in the database and the monitoring dashboard updates in real time.

The Three AI Agents

Each agent is a specialized microservice trained on different aspects of a request:

🔍 Agent 1	🦠 Agent 2	👤 Agent 3
Anomaly Detection	**Payload Analysis**	**Behavior Analysis**
Model: Isolation Forest	**Model: Character-level CNN**	**Model: LSTM (sequence model)**
Analyzes: Statistical patterns — request length, header entropy, character ratios	Analyzes: Raw payload content — scans for SQL, XSS, command injection signatures	Analyzes: User behavior over time — endpoint access sequences, session patterns
Output: Anomaly score 0–1 + reason	Output: Attack type + highlighted suspicious tokens	Output: Behavior score + sequence anomaly

Confidence-Aware Voting: How Decisions Are Made

Unlike traditional ensemble systems that give every model an equal vote, our system weights each agent's vote by:

📈 Threat Score(How suspicious?)	🎯 Confidence(How sure is the agent?)	⚖️ Historical Weight(How accurate recently?)

The weights update automatically every 1,000 requests — agents that perform well get more influence, and agents that underperform get less. The final decision thresholds:

Score < 0.3✅ ALLOW	Score 0.3–0.7⚠️ FLAG for Review	Score > 0.7 🚫 BLOCK

4. System Architecture

The system is built as a set of independent microservices, each with a clear responsibility. This design makes it easy to scale, maintain, and improve individual components without affecting the rest of the system.

Component Overview

Component	Technology	Layer	Purpose
API Gateway	Spring Boot	Ingestion	Receives requests, extracts features, enforces rate limits
Fast Filter	Python / Redis	Ingestion	Rule-based pre-filter; lets 80% of normal traffic skip AI analysis
Event Bus	Apache Kafka	Streaming	Passes suspicious requests to AI agents asynchronously
Anomaly Agent	Python / Isolation Forest	AI	Detects statistical outliers in request structure
Payload Agent	Python / CNN	AI	Scans payload content for attack signatures
Behavior Agent	Python / LSTM	AI	Monitors user session sequences for suspicious patterns
Decision Engine	Golang	Decision	Aggregates agent votes using confidence-weighted scoring
XAI Generator	Python	Explanation	Produces human-readable explanations for every decision
Cache & Session	Redis Cluster	Storage	Caches decisions and stores user session state
Audit Database	PostgreSQL	Storage	Persists all requests, decisions, and explanations
Dashboard	React.js	Monitoring	Real-time visualization of threats, metrics, and alerts

Data Flow Summary

Below is the full journey of a request through the system — from arrival to final decision:

Path for Normal Traffic (80%)	Path for Suspicious Traffic (20%)
Gateway → Fast Filter → ALLOW Processed in under 5ms. No AI involved. Result cached in Redis for future identical requests.	**Gateway → Kafka → 3 Agents → Vote → XAI → Decision** All three agents run in parallel. Total target: under 200ms end-to-end.

5. Explainability & Monitoring

Explainable AI (XAI) — Why Every Decision Is Justified

A key innovation of this system is that every security decision comes with a plain-English explanation. Security analysts should never see just 'BLOCKED' — they need to understand why.

Real-Time Monitoring Dashboard

A React-based dashboard provides live visibility into the system. All data updates in real time via WebSocket connections.

Dashboard Widget	What It Shows
Live Request Counter	Shows total requests processed, allow/block ratio in real time
Attack Type Breakdown	Bar chart showing distribution: SQLi, XSS, Command Injection, etc.
Latency Timeline	Line chart tracking response times — flags if latency exceeds 200ms
Agent Performance	Stacked area chart showing each agent's accuracy over time
Recent Decisions Feed	Live table with each blocked request, score, and XAI explanation
System Health Indicators	Status lights for all microservices, Kafka, Redis, and database
Manual Override Panel	Analysts can review flagged requests and approve or block manually

6. Technology Stack

Area	Technology	Reason for Choice
API Gateway	Spring Boot (Java)	Industry-standard, high-throughput HTTP handling with built-in security features
AI Agents	Python + TensorFlow / Scikit-learn	Best library ecosystem for ML models (CNN, LSTM, Isolation Forest)
Event Bus	Apache Kafka	High-throughput, fault-tolerant async messaging between microservices
Decision Engine	Golang	Low latency, compiled language ideal for performance-critical aggregation logic
Cache & State	Redis Cluster	Sub-millisecond read/write for caching decisions and rate limiting
Database	PostgreSQL	Reliable relational database for audit logs and historical analysis
Dashboard	React.js + Recharts	Real-time interactive UI with WebSocket support for live metrics
Monitoring	Prometheus + Grafana	Industry-standard observability stack for metrics and alerting
Containerization	Docker + Docker Compose	Easy local development and consistent deployment across environments